

Sigilix Engineering
Measurement Methodology, Product Architecture, and Token Economics

Daniel Martinez Julio, CEO and Founder
Sigilix

Abstract

This document is the engineering companion to the Sigilix benchmark report of June 2026. The report carries the numbers; this companion explains three things an evaluation table cannot: (a) the measurement system that produces those numbers, built so that they survive adversarial scrutiny, including our own; (b) the product surface the review engine opens up that pure review-bots structurally cannot offer—a CLI coding agent and a repository-grounded chatbot, both running on context the review loop has already earned and verified; and (c) the economic argument underneath both. A language model working through Sigilix does not pay to rediscover the codebase on every task, because Sigilix already holds that context—verified, indexed, and continuously refreshed by the review work the team is paying for anyway.

Contents

1	The measurement system: why the companion numbers deserve trust	3
1.1	A fixture designed to punish the failure mode that matters	3
1.2	Single runs are noise; we publish bands	3
1.3	Audit before trust — in both directions	3
1.4	The rules of honest comparison	4
1.5	Reproducible offline, at the seam users actually see	4
2	The believability algorithm: architecture, not prompt	4
2.1	Before the model: deterministic evidence first	5
2.2	Around the model: provenance contracts	5
2.3	After the model, before posting: refutation and execution	5
2.4	After posting: receipts the customer sees, and memory that learns	6
2.5	Why this is a position, not a feature	6
3	Beyond the review comment: what the engine earns, and the two lanes it opens	6
3.1	Lane one: the sigilix CLI	7
3.2	Lane two: the dashboard chatbot	7
3.3	Why competitors cannot follow easily	8
3.4	Who this serves	8
3.5	Status, stated plainly	9
4	Token economics: why the grounded call wins	9
5	What to hold us to	11
	References	11

1 The measurement system: why the companion numbers deserve trust

Most published AI-code-review numbers are marketing artifacts: a favorable run on an undisclosed fixture, scored by the vendor, reported as a point estimate. We decided early that our differentiator — believability — would be worthless if our own evaluation discipline did not hold to a higher standard than our product claims. So before tuning the engine, we built the apparatus that measures it. Four properties of that apparatus matter.

1.1 A fixture designed to punish the failure mode that matters

The benchmark fixture is not scraped or synthetic. It is a hand-constructed, deliberately dense pull request — 20 planted bugs across difficulty tiers (from deterministically-detectable up to intent-level subtleties like an uncapped refund and a read-modify-write race), packed into 281 lines of fullstack payments/refunds code.

Alongside the bugs sit **4 trap patterns**: code engineered to *look* dangerous while being genuinely correct. A string-built `IN (...)` clause whose placeholders are in fact fully parameterized. A query that looks like an authorization hole but is correctly scoped to the authenticated customer. Money arithmetic that looks like a float bug but is integer-cents throughout. These traps exist because the dominant failure mode of AI reviewers is not missing bugs — it is crying wolf, and teams tune out a reviewer that flags correct code. A reviewer that flags a trap takes a precision hit in our scoring; a reviewer that stays silent on one earns nothing. You cannot score well on this fixture by being noisy.

The ground truth was verified twice, by independent methods: first an independent blind bug hunt over the fixture by a reviewer with no access to the planting notes, then an adversarial audit of every individual label — which, notably, *changed* the labels: one pattern originally planted as a trap was reclassified as a real vulnerability, and several genuine unplanned bugs found by the blind hunt were promoted into the ground truth. The label set bears scars from its own verification process. That is what we mean by twice-verified.

1.2 Single runs are noise; we publish bands

LLM-based reviewers are stochastic. We measured this directly: on one *identical* diff, evaluated repeatedly with no changes whatsoever, we observed a competing reviewer swing from 7/20 to 10/20 across runs — a 15-point recall spread from sampling variance alone. Any vendor reporting a single-run score is, knowingly or not, reporting a die roll.

Our protocol therefore evaluates every configuration with **K-repeated runs** (**K = 12**) under a controlled harness, and we publish the run-to-run band, not the best draw. In the companion report, Sigilix’s controlled band is stated next to the live single-probe number, and the live number happens to sit at the *bottom* of the band — we report it as the headline anyway, because it is what production posted that day. Every score in the companion report passed through every stage of this pipeline: the twice-verified fixture, K-repeated runs on the identical diff under the controlled harness, matching of findings to ground truth (one row per root issue), a hand audit of every unmatched finding before it is called a false positive, and publication of the band rather than the best run.

1.3 Audit before trust — in both directions

Automated matching of a reviewer’s free-text findings to ground-truth rows is itself error-prone, so no unmatched finding is recorded as a false positive until a human has read the actual posted

comment against the actual code. This step is not ceremonial: on **five separate occasions** the hand audit corrected our own reported score — in both directions. Genuine catches that the matcher had mis-scored as false positives were restored; findings that were really coverage observations rather than bug claims were carved out of the precision denominator; and conversely, claims that flattered us did not survive contact with the diff. A measurement process that only ever finds errors in its favor is not a measurement process. Ours has a documented history of lowering our numbers when the evidence said to.

1.4 The rules of honest comparison

The companion report compares our measured numbers against competitor figures, and the comparison is governed by fixed rules, stated in the report itself:

- **Sourcing is labeled on every row.** Our column is measured on our fixture; competitor columns are the vendors’ own published figures from their own reports (CodeRabbit, 2026; Greptile, 2025) or from the independent Martian benchmark tracks (Martian, 2026). We never present the two as a same-fixture comparison, because they are not one.
- **Methodologies are attributed per track.** An acceptance-rate telemetry figure such as the Martian online track headlined by CodeRabbit (2026), a planted-bug catch-rate figure such as Greptile’s (2025), and a detection-coverage figure are different quantities; we cite each in its own regime and exclude from the headline table anything that is not a genuine recall/precision pair.
- **No shading where bands overlap.** Where our measured value falls inside a competitor’s published band — as our recall currently does inside Greptile’s (Greptile, 2025) — the row is claimed by no one, including us. Shading marks only clear leads, and the rule is symmetric: a published figure that clearly beat ours would carry the shading itself.

1.5 Reproducible offline, at the seam users actually see

The entire evaluation runs offline through deterministic replay harnesses — no live GitHub dependency, no production traffic required — executing the *exact production ensemble code path*, not a laboratory approximation of it. Crucially, the harness measures through the full post-synthesis stage, including the restoration floor that re-admits specialist findings the synthesis step would otherwise drop. That seam is what determines which comments actually get posted on a customer’s pull request; an evaluation that stops upstream of it reports a number no user ever experiences. Anyone with the fixture and the harness can re-derive every Sigilix number in the report.

2 The believability algorithm: architecture, not prompt

The most consequential contrast in the companion report is not the recall row. It is this: across the published field, false-positive rates of 45–74% are normal — those are the bands the independent benchmark tracks record for shipping products (Martian, 2026) — while our measured rate on the twice-verified fixture is 6.7%, with zero hallucinated findings. A gap of that shape is not produced by a stronger base model or a sharper prompt, and I want to be precise about why, because this section describes the part of Sigilix I consider hardest to copy. The field’s pipelines are, almost without exception, two stages: model → comment. A diff goes in, a language model speaks, and what it says gets posted. Ours is five: model → evidence → gates → receipts → memory. Believability is not a feature bolted onto that pipeline; it *is* the pipeline — an architecture that runs through every

finding’s lifecycle: before any model executes, around the model while it reasons, after the model before anything posts, and after posting, where the customer’s own behavior trains it. Everything described below is shipped and live unless explicitly marked otherwise.

2.1 Before the model: deterministic evidence first

No specialist model in our ensemble is the first authority on what a diff contains. Before any model runs, a deterministic pass computes the evidence the models will be held to: secret scans, dependency-vulnerability lookups, abstract-syntax-tree rule hits, static-analysis findings, and documentation-coverage signals, assembled into per-review manifests. The specialists do not get to assert in the abstract that a credential is exposed or a dependency is vulnerable; they must cite against the manifest. This inverts the usual division of labor. In a model→comment pipeline the model is asked to be detector, judge, and witness simultaneously; here the deterministic layer is the witness, and the model’s job narrows to interpretation — the job language models are actually good at.

2.2 Around the model: provenance contracts

The synthesis stage — where specialist candidates are merged into the review that posts — is where hallucination would enter, so it is bound by structural contracts rather than instructions. Every synthesized finding must cite the identifiers of the real specialist findings it derives from, and a post-synthesis structural check drops any finding whose citations do not resolve. The consequence is mechanical, not probabilistic: the synthesizer cannot introduce a finding no specialist produced, because a fabricated finding has nothing to cite and is removed by a check that consults no model at all. This is the precise reason the hallucinated-finding count in the companion report is zero rather than merely low.

The contracts are then specialized per domain. A security finding at high severity must carry a CWE identifier and an evidence identifier copied verbatim from the scanner manifest — or an explicitly justified statement of why no manifest entry exists. A testing finding must name the exact test files inspected. A performance finding must identify a concrete hot path or state a complexity estimate. In each case the contract converts a category of plausible-sounding fabrication into a missing required field.

2.3 After the model, before posting: refutation and execution

Findings that survive the contracts then face two adversarial stages. The first is refutation. Each specialist is required to attempt to refute its own candidates against the code, and a candidate cleared this way is recorded with the exact mitigating token quoted from the source. The same discipline binds the synthesizer in the opposite direction: it may drop a high-severity specialist finding only by quoting a proof-like contradiction from the evidence; a generic “looks fine” drop is restored deterministically by the post-synthesis floor — the same seam Section 1.5 insists on measuring through. Every drop and every softening is itself recorded with its reason, so each review carries a provenance ledger of what was claimed, what was challenged, and on what grounds anything was removed.

The second stage is execution. Findings that admit a runtime check are actually run, and findings the execution refutes are demoted before anything posts — the model’s opinion loses to the program’s behavior. The execution evidence itself is hardened: runner receipts are cryptographically bound to the tuple (repository, commit SHA, run identifier, attempt) with single-use enforcement, so a receipt can neither be replayed against a different commit nor forged after the fact.

2.4 After posting: receipts the customer sees, and memory that learns

The evidentiary class of every claim is then surfaced, not buried. Each posted finding renders a proof tier — VERIFIED for execution-backed claims, GROUNDED for claims carried by deterministic evidence, MODEL for claims resting on model judgment alone — so the reader knows, per finding, what kind of thing they are being asked to believe. Receipts, not vibes, in the product surface itself.

And the lifecycle does not end at the comment. Every dismissal a team issues feeds a per-repository corpus, and recurring rejected finding classes are softened in subsequent reviews; calibrated demotion is the deliberate next stage, gated on accumulated data rather than guessed at. This makes the believability system the only part of the product a customer trains simply by disagreeing with it.

2.5 Why this is a position, not a feature

The loop closes with measurement. A false-positive-rate endpoint and validation-metrics telemetry track believability in production, which makes it a measured, regression-tested property of the system rather than an aspiration — the same audit-before-trust discipline as Section 1, applied to the live product. Read against that, the published 45–74% false-positive band (Martian, 2026) is best understood not as a model-quality gap but as the visible symptom of pipelines that have none of the layers above: no deterministic evidence layer, no provenance contract, no execution verification, no cryptographic receipts, no dismissal learning. A better model narrows nothing here, because the layers are not made of model; they have to be built, hardened, and measured one at a time, and each of ours was.

Two more sentences belong in this section. The strategic one: the believability data this architecture accumulates — the manifests, the provenance ledgers, the receipts, the adjudicated dismissals — is exactly the context layer the CLI and chatbot lanes of Sections 3 and 4 are built to serve, so every review that runs makes the agent platform more trustworthy; the moat compounds. And the honest one: these gates cost recall when they are mistuned — a floor set too low or a contract too strict suppresses true findings — which is why every new gate ships shadow-first, and why the harness of Section 1 measures exactly that trade at the seam where it occurs.

3 Beyond the review comment: what the engine earns, and the two lanes it opens

Greptile and CodeRabbit are review-bots in the precise sense: the product’s output is the PR comment, and the product ends there. Whatever their models learned about your codebase while reviewing is discarded as exhaust.

Our architecture inverts this. The review loop is not just a comment generator — it is a process that continuously *earns* a verified, machine-usable understanding of each repository, as a byproduct of work the team is already paying for:

- **A vector + lexical index** of the repository, kept current as reviews flow.
- **A code and dependency graph** — which symbols call which, what depends on what — used today for retrieval during review.
- **A trust ledger:** which findings were verified real and which the team dismissed — the dismissal corpus. This is ground truth about *this team’s* signal/noise boundary that no fresh model session possesses.

- **Per-repository review memory:** recurring conventions, past verdicts, claims that have already been adjudicated.
- **Security evidence manifests:** where the security posture has been probed, with the receipts.

What distinguishes this corpus from an embedding cache is that much of it is *verified*: findings carry evidence trails, the trust ledger records human adjudication, and the believability machinery of Section 2 that gates what we post is the same machinery that gates what enters the corpus. Two product lanes are being built on top of it; Figure 1 shows how they relate.

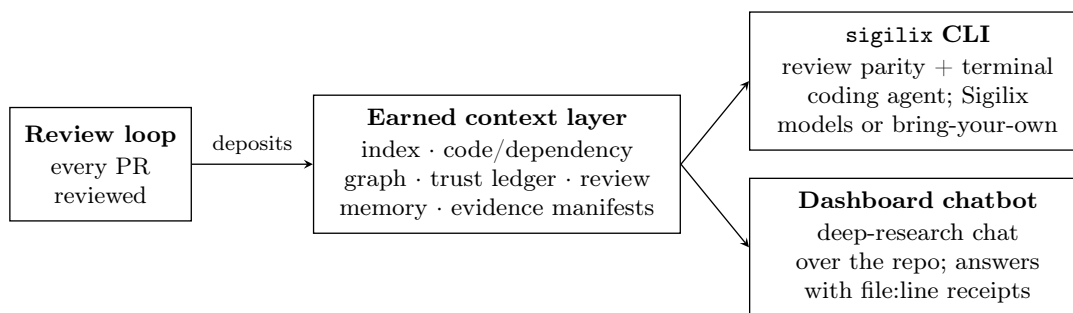


Figure 1: The review-to-context-to-products pipeline. The review loop the team already pays for deposits verified context; the CLI and chatbot lanes draw on it.

3.1 Lane one: the sigilix CLI

The CLI has two parts. The first is table stakes: **review parity** — `sigilix review -diff`, `-staged`, and so on, matching what competitors’ CLIs do, backed by the same engine whose precision is measured in the companion report.

The second part is the differentiator: a **terminal coding agent** in the style of Claude Code or Codex CLI, where developers code *with* Sigilix models — or bring their own (OpenRouter, Codex CLI, Claude Code) with transparent usage pricing. The bring-your-own case is the telling one: the value being sold is not our chat loop, it is the context infrastructure underneath it. A customer’s own model, working through Sigilix, fetches the believability data, the KV stores, and the indexed repository context that the review loop has accumulated — and inherits the precision that comes with them.

One engineering fact makes this lane cheap to ship rather than aspirational: **the engine already runs decoupled from its deployment platform, in plain Node** — proven, not asserted, because our offline evaluation harness (Section 1.5) executes the exact production ensemble code outside the platform today. The CLI is a packaging problem, not a rewrite.

3.2 Lane two: the dashboard chatbot

The same earned context powers a deep-research chat surface over the indexed repository. Ask “where do refunds get capped?” and the answer is not a model’s recollection — it is assembled from the code graph and the index, and it arrives with `file:line` receipts, in the same evidence-grounded style as our review findings. For an engineering leader, this turns the review subscription into a queryable, current model of the codebase. Neither incumbent has the substrate for this, because their products discard at the end of each review precisely the thing this lane is built from.

3.3 Why competitors cannot follow easily

The obvious competitive response to the two lanes is “we will ship a CLI too,” and I want to explain why that response misses the structure of the situation. The incumbents approach from two sides, and neither side can cross easily.

From the review side: Greptile’s and CodeRabbit’s products end at the PR comment, and their CLI offerings are review-parity at best — the same comment, delivered to a terminal. Nothing in that motion produces an agent, because nothing in it retains what the review learned.

From the agent side: Cursor and Copilot field genuinely capable terminal and IDE agents, but those agents are **context-blind per session**. They hold no verified record of what is real versus dismissed in a given codebase — they cannot, because they never do the reviewing. The adjudicated history that separates this team’s true signal from its known noise simply does not exist anywhere in their stack; each session starts from the raw repository and a blank slate.

The moat is therefore not the chat loop on either side. It is the data asset the review loop earns — the index, the code and dependency graph, the trust ledger, the dismissal corpus, the security evidence manifests enumerated at the head of this section — and the direction of travel across that asset is asymmetric. **A reviewer can become an agent platform**, because reviewing every pull request continuously deposits exactly the verified substrate an agent needs. **An agent cannot easily become a reviewer with a believability record**, because the record accrues only from adjudicated review work performed over time on the customer’s actual pull requests — there is no shortcut that synthesizes a dismissal corpus or a trust ledger from a cold repository.

Our bring-your-own-model support (OpenRouter, Codex CLI, Claude Code) makes the point sharper rather than weaker: we win even when the customer brings a competitor’s model, because the value is the context layer, not the chat loop. If the model is interchangeable and the outcome still favors working through Sigilix, the advantage sits precisely in the part competitors cannot copy quickly.

3.4 Who this serves

The two lanes serve two audiences, and the second expands who the product is for.

Engineers. The CLI gives them review parity in the terminal (`sigilix review -diff, -staged`) and a coding agent whose fixes land with `file:line` receipts grounded in the verified graph rather than in a session’s guesswork. Because the dismissal corpus travels with the context, the agent does not re-litigate patterns the team has already adjudicated and dismissed — the noise boundary their review history paid to establish is enforced in the editor loop too. And their existing model subscriptions work via BYO: adopting the agent does not mean abandoning the models they already pay for.

Non-technical roles. The dashboard chatbot makes a codebase legible without reading code, and this is where the audience widens beyond engineering. A product manager asks “is the refund cap enforced anywhere?” and gets a graph-grounded answer with `file:line` receipts. A founder or compliance lead asks for the security posture and gets an answer backed by the same evidence manifests the reviewer uses. A new hire onboards by interrogating the repository conversationally instead of shoulder-tapping senior engineers. Every answer carries the same believability grounding as a posted review finding — receipts, not vibes. The business consequence fits in one sentence: this moves the product beyond per-engineer seats toward whole-team surface area.

3.5 Status, stated plainly

The engine, the index, the graph, the trust ledger, and the evidence machinery are in production today — they are how the benchmark numbers get produced. The CLI and chatbot lanes are the build-out on top: scoped, on the near-term roadmap, and deliberately described in this document as architecture and direction rather than as shipped product. The same discipline applies to the two subsections above: the competitive asymmetry of Section 3.3 is a claim about structure, not about market outcome, and the audiences of Section 3.4 describe the surface we are building toward, not users we are counting today. We are not claiming users; we are claiming that the hard part — the verified context layer and a decoupled engine — already exists and is measured.

4 Token economics: why the grounded call wins

This is my core thesis: an argument about cost structure, not model quality.

Today’s coding agents are expensive for a structural reason: **they rediscover context every session**. Give a context-blind agent a task — say, “cap refunds at the amount actually paid” — and watch what it spends before the first productive edit: it greps for **refund**, opens files, follows imports, re-derives the dependency structure, re-discovers that this codebase keeps money in integer cents on the wire, re-learns which patterns this team considers idiomatic. Session after session, task after task, the same facts are re-purchased at full token price — because the agent’s understanding evaporates when the session ends. The industry’s answer has been bigger context windows, which raises the ceiling on how much can be re-read per session without changing the fact that it is being *re-read*.

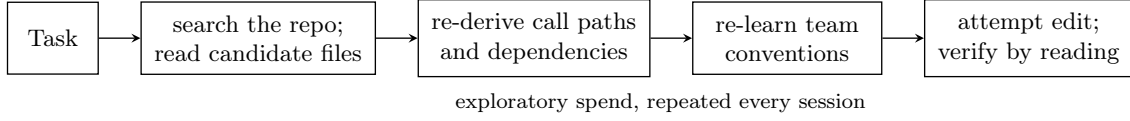
My answer is that the context should not need rediscovering, because the organization already paid to establish it. The review loop runs on every pull request; as a byproduct (Section 3) it maintains the index, the graph, the conventions, the team’s adjudicated signal/noise boundary, and the security posture map — verified and continuously refreshed. An LLM working through Sigilix therefore makes **one precise call with the right context attached**: the relevant subgraph, the applicable conventions, the prior verdicts on this exact pattern. And a customer bringing their own model gets the same mechanics — their model fetches Sigilix’s believability data, KV stores, and indexed contexts instead of spelunking. Figure 2 sketches the contrast in flow terms; Table 1 itemizes it.

Two consequences follow, and they point the same direction:

- **Cost: tokens stop being the cost driver.** The exploratory phase — the bulk of a context-blind session — is amortized into the review loop the team already runs. What remains per task is the grounded call itself.
- **Precision rises at the same time.** This is the part that breaks the usual cost/quality trade-off: a context-grounded fix lands where a context-blind agent guesses. The same earned context that eliminates the exploratory spend is what makes the single call accurate — the agent is not choosing between cheap and right.

We will keep the quantitative version of this thesis honest the same way we keep the benchmark honest: by measuring it before publishing it. The illustrative contrast above is labeled as such. The one hard cost number we will put in this document, because it traces to the record: **this week’s synthesis-model swap cut the per-call cost of our most-frequently-made model call by roughly 4× — while *improving* measured quality**, winning its evaluation bake-off on every

Context-blind agent, per task:



Through Sigilix, per task:

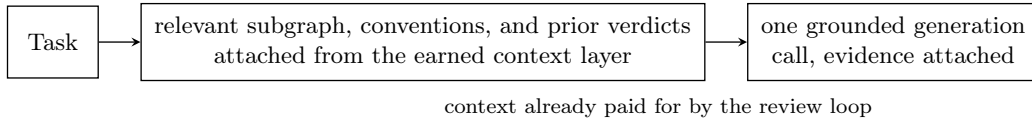


Figure 2: The token-economics contrast. A context-blind agent re-purchases repository context on every session; a model working through Sigilix makes one grounded call with context the review loop has already earned and verified.

Table 1: Where tokens go per task: context-blind agent versus a Sigilix-grounded call

Context-blind agent, per task	Through Sigilix, per task
Search the repo; open and read candidate files	Fetch the relevant subgraph from the code/dependency graph
Re-derive call paths and dependency structure	Already held — maintained by the review loop
Re-learn conventions (e.g. integer-cents money)	Attached from review memory, pre-verified
Guess which concerns are real vs. noise here	Trust ledger supplies the team’s adjudications
Then attempt the edit — and verify by more reading	One grounded generation call, evidence attached

Note. Illustrative contrast, not a measured benchmark. The left column’s exploratory calls dominate session cost and are repeated every session; the right column amortizes them across the review work already being paid for.

axis before it shipped. That is one data point, not the thesis proven; but it is the cost discipline that runs through the stack, applied at the engine’s hottest call site, with the quality gate enforced first.

5 What to hold us to

The honest-measurement posture is not a stylistic choice; it is the moat. A reviewer competes on being believed, and a vendor whose own numbers do not survive audit has already lost that competition. So: the benchmark claims in the companion report each carry their sourcing regime on the row; the believability mechanisms of Section 2 are, without exception, shipped and measured rather than roadmap, save the one explicitly marked as data-gated; the product claims in Section 3 are split explicitly into *in production today* versus *architecture and direction*; and the economic claims in Section 4 contain exactly one hard number, the one that traces to a gated, measured change. When the CLI and chatbot lanes ship, and when the token-economics thesis has measured numbers behind it, those numbers will be produced by the same apparatus described in Section 1 — bands, audits, and all. Hold us to that.

Sigilix Engineering · June 12, 2026 · companion to the Sigilix Benchmark Report (recall update), June 2026. Fixture details, scoring rules, and per-row sourcing for all benchmark figures referenced here appear in the report’s methodology note and appendix. Sigilix figures cited in this document are internal measurements taken in June 2026 and require no external citation; external benchmark figures are cited author–date and listed below.

References

- CodeRabbit. (2026, March 3). *CodeRabbit tops independent AI code review benchmark*. CodeRabbit Blog. Retrieved June 11, 2026, from <https://www.coderabbit.ai/blog/coderabbit-tops-martian-code-review-benchmark>
- Greptile. (2025). *AI code review benchmarks 2025* (evaluation period July 2025). Retrieved June 11, 2026, from <https://www.greptile.com/benchmarks>
- Martian. (2026). *Code review benchmark* (offline and online evaluation tracks; offline results updated April 11, 2026). GitHub. Retrieved June 11, 2026, from <https://github.com/withmartian/code-review-benchmark> (online leaderboard at <https://codereview.withmartian.com/>)